Code and Hardware Explained

https://gemini.google.com/app/ac1ba5059097e70a

User prompt: With the context you have can you understand what Im trying to do with this code?

Response: Yes, absolutely! Looking at the Python scripts (Bolgy_wolgy_code.py / badge code.docx) alongside your KiCad board design and schematic setup, the whole vision for this DEF CON badge comes together very clearly.

Here is a breakdown of what you are building and how your hardware and software work together:

🎯 The Core Concept

You are building an NSFW, furry-themed parody of the **Nintendo Switch logo** as a physical DEF CON badge.

- **The Main Badge ("Switch" Base):** Holds your microcontroller, audio speaker circuit, power supply, and a **2x3 female SAO socket**.
- **The SAO ("Shitty Add-On" Plug-in):** Designed as the penis piece that physically plugs into the main badge.
- **The Interaction:** When someone plugs the SAO into the main badge, the physical pins short circuit a detection line. This triggers the hardware to play the signature **Nintendo Switch click sound** over the speaker and run a script tracking the interaction.

💻 What the Code is Doing Step-by-Step

1. Hardened State & Counter Management (Bolgy_wolgy_code.py)

In Bolgy_wolgy_code.py, you set up a dataclass (BadgeState) that reads and writes to a persistent file (badge_test.dat):

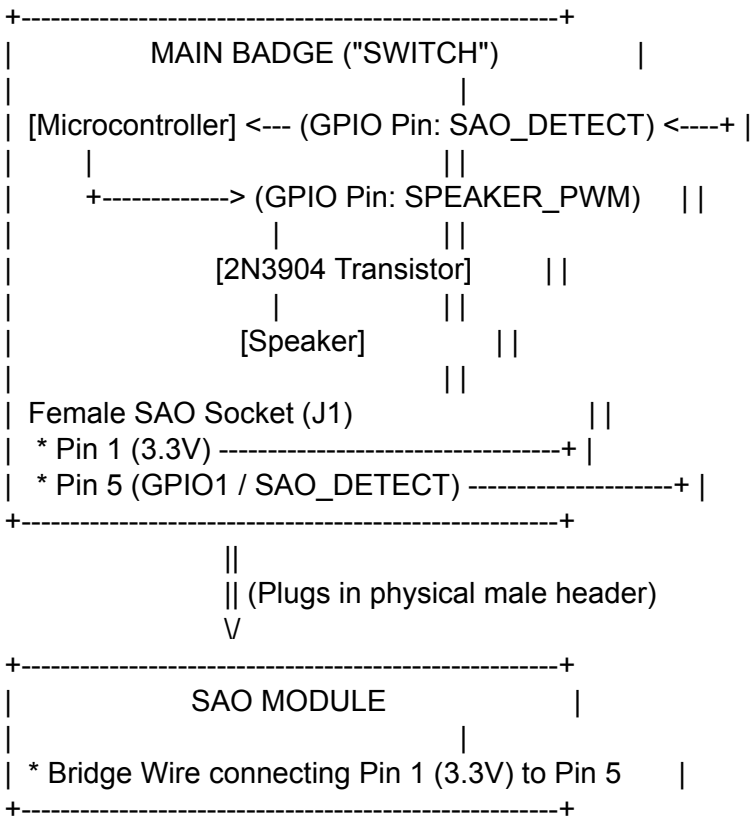
- **script_runs (Global Counter):** Tracks the total number of times the SAO has been connected/triggered.
- **counts (Action Breakdown):** Tracks individual counters for each specific encounter type so you know how many times each specific act has occurred.
- **JSON Fallback:** If badge_test.dat is missing or corrupted, it safely initializes a fresh counter set without crashing the microcontroller.

2. Event Triggering & Weighted Encounters

When trigger_random_action() runs upon SAO connection:

1. It picks a random encounter from your RESPONSES dictionary:
    - **yiff**
    - **blow_job**
    - **cream_pie**
    - **paw_job**
2. It increments the total run count (script_runs += 1) and the specific action count (counts[action] += 1).
3. It prints out the action name, the adult descriptive text, and summary stats showing how many times that specific event (and overall events) have occurred.
4. It updates badge_test.dat so the data persists even if the badge power cycles at the con.

🛠️ How It Ties Into Your KiCad Hardware Design



1. **Detection:** When the SAO plugs in, 3.3V from **Pin 1** flows into **Pin 5**. The microcontroller sees SAO_DETECT go **HIGH**.
2. **Audio Trigger:** Your MCU code catches that signal interrupt, triggers PWM audio to play the Switch click sound through transistor Q1 to speaker LS1.
3. **Execution:** The Python script runs trigger_random_action(), outputting the encounter result and saving the updated count to disk.

User prompt: Can you put this all in a write-up?

Response: Project Overview: DEF CON

"Switch" Parody Badge & SAO

Executive Summary

This project consists of a physical, interactive electronic badge designed for DEF CON. The badge functions as an NSFW, furry-themed parody of the Nintendo Switch logo.

It features a two-part hardware architecture:

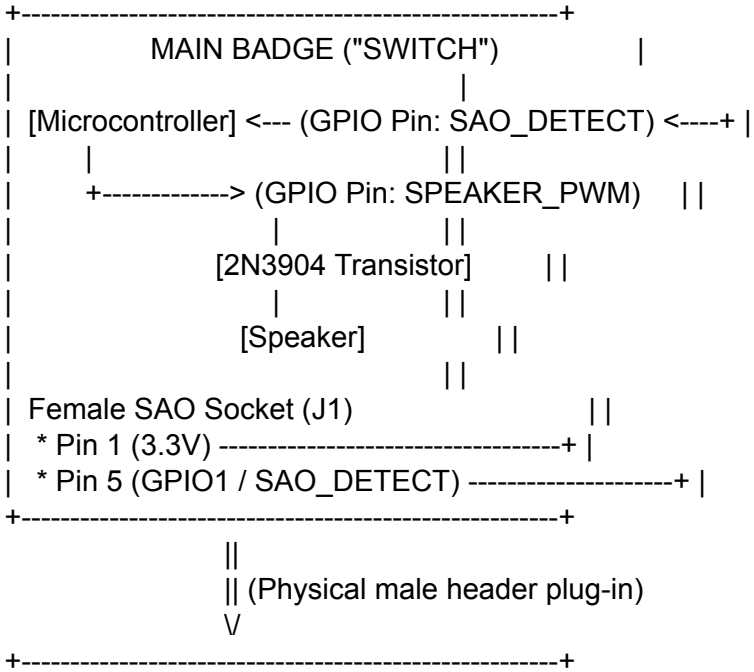
1. **The Main Badge ("Switch" Base):** Holds the microcontroller, speaker driver circuit, power regulation, and a female SAO socket.
2. **The SAO ("Shitty Add-On" Plug-In):** Formatted as the penis artwork piece that physically connects to the main badge.

When plugged in, mechanical pins on the SAO complete a circuit , signaling the badge microcontroller to play the signature **Nintendo Switch click sound** via PWM audio while executing Python firmware that logs the interaction, updates persistent state storage , and returns descriptive encounter text.

🛠️ Hardware Design & KiCad Implementation

1. Circuit Architecture

- **Connection Detection:** Standard SAO connector v1.69/v2 layout (2x3 pin header, 2.54mm pitch). Pin 1 provides +3.3 V power , Pin 2 connects to ground , and Pin 5 acts as SAO_DETECT.
- **Detection Mechanism:** On the base badge, Pin 5 (SAO_DETECT) is pulled LOW to 0 V via a 10 kΩ pull-down resistor. On the SAO module, Pin 1 (+3.3 V) is directly bridged to Pin 5. Upon connection, Pin 5 goes HIGH (+3.3 V), triggering a hardware interrupt on the microcontroller.
- **Audio Driver:** A microcontroller PWM pin (SPEAKER_PWM) drives the base of a 2N3904 NPN transistor via a 1 kΩ resistor. The transistor acts as a low-side switch controlling an 8 Ω micro speaker/piezo buzzer connected to +3.3 V.



```
|                  SAO MODULE                    |
|                                                |
|  * Bridge wire connecting Pin 1 (3.3V) to Pin 5      |
+------------------------------------------------+
```

2. KiCad Netlist & Component Bill of Materials

- J1: Conn_02x03_Odd_Even (2x3 Female SAO Socket Header)
- J2 / J3: Conn_01x07 (MCU Breakout Headers)
- R1: 10 kΩ Resistor (Pull-down on SAO_DETECT)
- R2: 1 kΩ Resistor (Transistor Base Limiter)
- Q1: 2N3904 NPN Transistor (TO-92 Package)
- LS1: 8 Ω Micro Speaker / Piezo Buzzer

💻 Microcontroller Firmware (Bolgy_wolgy_code.py)

The firmware uses Python (CircuitPython/MicroPython compatible) utilizing dataclasses and persistent file system reads/writes to track state across power cycles at the event.

Source Code

```
from dataclasses import dataclass, field
from pathlib import Path
import json
import random

# Persistent storage file location
DATA_FILE = Path("badge_test.dat")

# Configurable responses dictionary
RESPONSES = {
    "yiff": "You just got fucked hard in the stair-well",
    "blow_job": "You found a big dick tiger at a room party, his cock almost made you choke",
    "cream_pie": "You just finished fucking in the bathroom, he left you dripping",
    "paw_job": "You just got a paw_job under the table",
}


@dataclass
class BadgeState:
    script_runs: int = 0
    counts: dict[str, int] = field(
        default_factory=lambda: {action: 0 for action in RESPONSES}
    )

    @classmethod
    def load(cls, filepath: Path) -> "BadgeState":
        """Loads state from JSON file; defaults if missing or invalid."""
        if not filepath.exists():
            return cls()
```

```
        try:
            data = json.loads(filepath.read_text())
            script_runs = data.get("script_runs", 0)
            counts = {
                action: data.get("counts", {}).get(action, data.get(action, 0))
                for action in RESPONSES
            }
            return cls(script_runs=script_runs, counts=counts)
        except (json.JSONDecodeError, OSError):
            print("Warning: Could not parse badge data file. Starting fresh.")
            return cls()

    def save(self, filepath: Path) -> None:
        """Saves state back to JSON file."""
        payload = {
            "script_runs": self.script_runs,
            "counts": self.counts,
        }
        filepath.write_text(json.dumps(payload, indent=2))

    def trigger_random_action(self) -> tuple[str, str]:
        """Picks a random action, increments counters, and returns result."""
        action = random.choice(list(RESPONSES.keys()))
        self.counts[action] += 1
        self.script_runs += 1
        return action, RESPONSES[action]


def main():
    badge = BadgeState.load(DATA_FILE)
    action, message = badge.trigger_random_action()

    print(action)
    print(message)
    print(f"{action} has occurred {badge.counts[action]} time(s).")
    print(f"You've gotten yiffed {badge.script_runs} time(s).")

    badge.save(DATA_FILE)


if __name__ == "__main__":
    main()
```

📁 Repository File Structure Reference

```
DEFCON-Switch-Badge-Bolgy-Wolgy/
├── Hardware/
[cite_start]│   ├── DEFCON_Switch_Badge.kicad_pro   # KiCad Project File [cite: 715]
[cite_start]│   ├── DEFCON_Switch_Badge.kicad_sch   # Main Badge Schematic [cite: 715]
[cite_start]│   ├── DEFCON_Switch_Badge.kicad_pcb   # PCB Layout & Edge.Cuts [cite: 715]
```

```
[cite_start]│   ├── Switch_Badge_Art.kicad_mod      # Converted Silkscreen Footprint [cite: 715]
[cite_start]│   └── Gerbers/                       # Manufacturing Drill/Gerber files [cite: 724]
├── Artwork/
│   ├── bolgy wolgy.JPG                # Main Switch Parody Art
│   └── photo_2026-08-03_13-50-09.jpg   # SAO Insertion Visual Target
├── Firmware/
[cite_start]│   ├── Bolgy_wolgy_code.py            # Main Badge Execution Logic [cite: 715]
[cite_start]│   └── badge_test.dat                 # Persistent State Storage JSON [cite: 16]
[cite_start]└── README.md                          # Project Description & Specifications [cite: 715]
```
